

AI WeatherWise — Intelligent Weather Forecasting & Insights

Description

AI WeatherWise is a REST-based backend system developed with Node.js and Express.js. MongoDB is used for persistent storage through Mongoose, while Google Gemini AI converts current weather information into easy-to-understand summaries and personalized suggestions. The service combines live weather data with AI-generated guidance so users can make better-informed plans from current environmental conditions.

The backend includes JWT-based authentication, bcrypt password protection, favorite-location management, and dedicated APIs for weather and recommendation features. A fallback mechanism keeps core API behavior available when an external API key is unavailable, while centralized error processing and request sanitization improve reliability.

Scenario-Based Case Study

Background

Alex regularly travels between cities and wants one convenient service for checking local weather and getting activity suggestions that match the current forecast.

Problem

- Managing several favorite cities can become inconvenient.
- Raw weather values do not always provide practical guidance.
- Users may want suggestions for outdoor activities and suitable clothing.
- Preferences need to be stored securely and remain available across sessions or devices.

Solution

AI WeatherWise acts as a central backend for retrieving weather information and producing AI-assisted recommendations. User and location data are stored securely, while Gemini is used to generate contextual summaries and suggestions from the available weather information.

Usage

- User: Registers/logs in, manages favorite cities, checks current weather, and requests AI insights.
- Admin/Developer: Checks API health, monitors registered users, and verifies weather/AI integrations.
- Public Weather User: Can consume supported weather information without maintaining an account.

Outcome

The system reduces the effort required to interpret weather metrics by presenting them with contextual AI guidance. Mongoose provides structured data handling, while the Gemini integration adds readable summaries and practical recommendations.

```
const mongoose = require('mongoose');

const locationSchema = new mongoose.Schema({
  city: {
    type: String,
    required: [true, 'Please provide a city name'],
    trim: true
  },
  country: {
    type: String,
    required: [true, 'Please provide a country name'],
    trim: true
  },
  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: [true, 'User reference is required']
  }
}, {
  timestamps: true
});

// Ensure a user cannot duplicate the exact same city and country in their favorites
locationSchema.index({ user: 1, city: 1, country: 1 }, { unique: true });

const Location = mongoose.model('Location', locationSchema);
module.exports = Location;
```

1. Software & Hardware Requirements

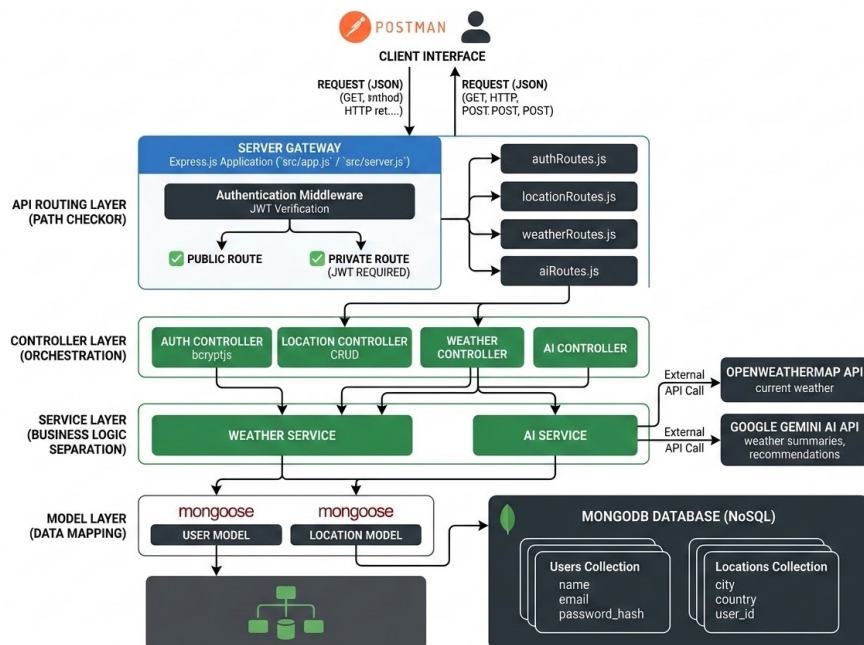
Software Requirements

- Operating System: Windows 10/11, macOS, or Linux.
- Node.js (v16 or later): Runs the backend application and server logic.
- npm (v8 or later): Manages project packages and dependencies.
- Gemini AI SDK: Connects the application with Google Gemini for natural-language insights.
- MongoDB: Stores user and location information as a NoSQL database.
- Postman: Used to send requests and verify API responses.
- Visual Studio Code or another suitable code editor.

Hardware Requirements

- Processor: Intel Core i5 (8th Gen or newer), AMD Ryzen 5, or equivalent.
- RAM: At least 8 GB; 16 GB is recommended for smoother concurrent development.
- Storage: At least 1 GB of free project space.

Epic 1: Project Architecture



System Architecture

The AI WeatherWise backend follows an MVC-oriented structure that separates application logic, data definitions, and request handling. Weather information is retrieved from OpenWeatherMap, while Gemini produces AI-based summaries and recommendations. User and favorite-location records are maintained through MongoDB and Mongoose.

Components Definition

- Interface Layer: Postman or similar API clients send requests to the available endpoints and receive JSON responses.
- Express Server: Receives HTTP requests, applies middleware, parses JSON data, handles CORS, and forwards requests to the correct routes.
- Authentication & Authorization Middleware: Reads JWT tokens, validates the authenticated user, and protects restricted endpoints.
- Service/Logic Modules: Keep domain-specific processing separate from routing and database code.
- Database Layer: Mongoose maps application objects to MongoDB collections and operations.
- AI Integration: The Gemini service converts current weather conditions into summaries and personalized recommendations.

Entity-Relationship (ER) Diagram Description

The data model links registered users with the cities they save as favorites.

User Entity

- `_id` — Primary key (ObjectId)
- `name` — Required string
- `email` — Required and unique string
- `password` — Required hashed string
- `role` — String with a default value of “reader”

Location Entity

- `_id` — Primary key (ObjectId)
- `user` — Reference to the User collection
- `city` — Required string
- `country` — Required string
- `createdAt` — Date with the current time as default

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/weatherwise');
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`Error connecting to MongoDB: ${error.message}`);
    process.exit(1);
  }
};

module.exports = connectDB;
```

Data Relationships & Key Features

Key Structural Mapping

- User → Location: One-to-Many. A single registered user can save several favorite locations.

Key Features

- Role-based route protection for restricted operations.
- CRUD support for favorite locations and related records.
- Categorization/filtering support where required by the application.
- Controlled moderation/status handling for protected content paths.
- Input sanitization to reduce XSS and NoSQL injection risks.

Roles and Responsibilities

Admin:

- Manages user accounts and monitors system/API activity.
- Oversees database integrity and operational logs.

Registered User:

- Adds and manages favorite locations.
- Requests current weather information.
- Uses Gemini-powered summaries and activity recommendations.
- Maintains their own account information.

User Flow

1. The user registers or signs in.
2. The user saves a city as a favorite.
3. The application retrieves current weather information.
4. The user requests an AI summary or recommendation.
5. The backend validates the JWT and returns the requested result.

MVC Architecture Pattern

The project uses an MVC-style separation to keep data, processing, and API routing responsibilities distinct.

```
const app = require('./app');
const connectDB = require('./config/db');

const PORT = process.env.PORT || 5000;

connectDB()
  .then(() => {
    app.listen(PORT, () => {
      console.log(`Server running on port ${PORT}`);
      console.log('Press Ctrl+C to terminate.');
```

Epic 2: Project Setup & Configuration

Model Layer (Data Blueprint)

Mongoose schemas define the structure of records stored in MongoDB. They specify fields, required values, defaults, and validation behavior before data is persisted.

Controller Layer (Execution Logic)

Controllers receive requests from the route layer, validate the incoming data, coordinate the required processing, interact with models/services, and return standardized JSON responses.

View Layer (API Interface)

Because this project is a headless backend, it does not use traditional HTML template views. The API routing layer acts as the interface between clients and controller functions.

Creating the Project Folder

- Create a folder named AI WeatherWise.
- Open the folder in Visual Studio Code.
- Initialize the Node.js project and add the required packages.

Project Initialization

Run the following commands in the VS Code terminal:

```
npm init -y
```

```
npm install express mongoose bcryptjs jsonwebtoken cors dotenv multer @google/genai
```

```
npm install --save-dev nodemon
```

Required Files

- index.js
- .env
- package.json
- .gitignore

Environment Configuration

Create a .env file in the project root and provide the required configuration values:

```
PORT=5000
```

```
MONGO_URI=your_mongodb_connection_string
```

```
JWT_SECRET=your_jwt_secret
```

```
GEMINI_API_KEY=your_gemini_api_key
```

http://localhost:5001/api/weather/bangalore

Save

No Environment

GET

http://localhost:5001/api/weather/bangalore

Send

Params

Authorization

Headers (10)

Body

Scripts

Settings

Code

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

Be

1 {

2 ... "email": "john@mail.com",

3 ... "password": "123456"

4 }

Body

Cookies

Headers (8)

Test Results

Status: 200 OK

Time: 305 ms

Size: 392 B

...

Pretty

Raw

Preview

JSON

...

1 [{"success": true, "data": {"city": "Bangalore", "temperature": 34, "humidity": 79, "windSpeed": 12, "condition": "Windy", "isMock": true}}]

Epic 3: Backend Development

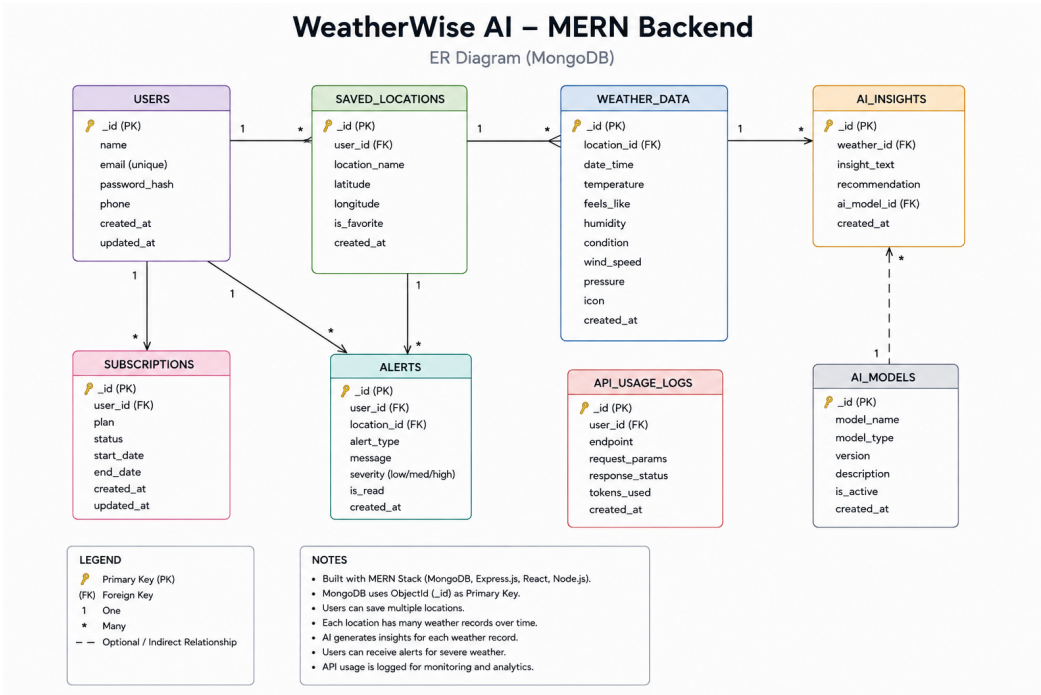
The AI WeatherWise backend is implemented with Node.js, Express.js, and MongoDB using a modular structure. Routes, controllers, middleware, models, and utility logic are separated so that the application remains easier to maintain and extend. The REST API provides authentication, weather-related operations, AI-powered recommendations, and administrative functions.

Backend Structure

Core System Script Implementations

1. Application Entry Point (index.js)

The index.js file acts as the starting point of the backend. It loads environment variables, creates the Express application, registers middleware and routes, establishes the MongoDB connection, and starts the server after the database connection is ready.



Epic 4: Database Configuration

API Testing Demo

1. Configure MongoDB

Install Mongoose in the Node.js project:

```
npm install mongoose
```

Create Database Connection

The backend should establish a successful MongoDB connection before processing application operations. The connection module is responsible for initializing communication between the application and the database.

Create Schemas & Models

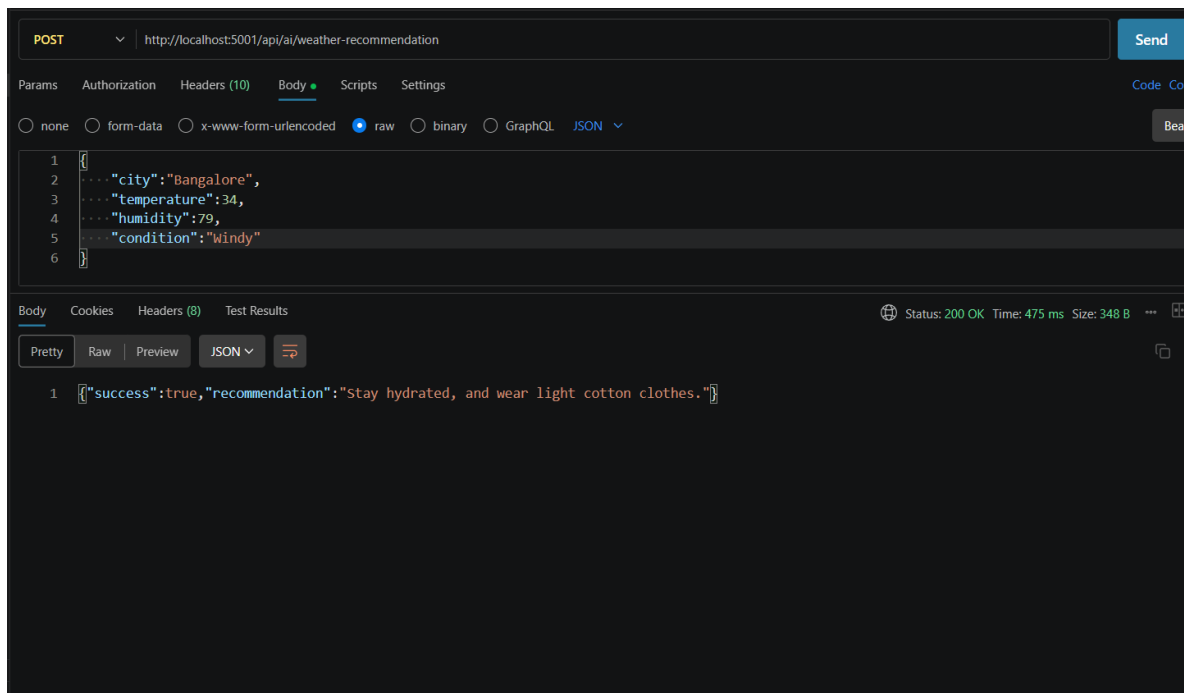
Define the User and Location schemas according to the fields required by the application, then create Mongoose models from those schemas.

Location Schema

User Schema

DB Connection

This module connects the application to MongoDB through Mongoose. Establishing the connection before serving requests ensures that database-dependent operations have an active data layer.



Epic 5: Project Execution

The AI WeatherWise backend can be run locally with Node.js. Before starting the application, MongoDB must be available and the environment variables must contain valid configuration values. API endpoints can be verified through Postman or Thunder Client.

Running the Backend Server

Install the project dependencies:

```
npm install
```

Start the development server:

```
npm start
```

Once execution is successful, the application connects to MongoDB and begins listening for incoming API requests.

API Testing

The main API tests covered in the project include registration, login, weather retrieval, and AI recommendation requests.

Register API

Login API

Weather API

Recommendation API

API Testing Video Link

API testing.mp4

Demo Video Link

Backend demo.mp4

CodeDrive

Code

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Please provide a name'],
    trim: true
  },
  email: {
    type: String,
    required: [true, 'Please provide an email'],
    unique: true,
    trim: true,
    lowercase: true,
    match: [
      /^\\w+([\\.-]?\\w+)*@\\w+([\\.-]?\\w+)*(\\.\\w{2,3})+$/,
      'Please provide a valid email address'
    ]
  },
  password: {
    type: String,
    required: [true, 'Please provide a password'],
    minlength: [6, 'Password must be at least 6 characters']
  }
}, {
  timestamps: true
});
```